Republic of Turkey

Kastamonu University

Faculty of Engineering and Architecture

Department of Computer Engineering

OBJECT-ORIENTED PROGRAMMING

PROJECT REPORT

Project Owner	İrem AKGÜN
School Number	234410151
Advisor	Ahmet Nusret ÖZALP
Course Code	BSM204
Date	30.05.2026

Hook Detection Tool

Inline Hook Detection Tool: An Object-Oriented C++ Demonstration Using RAII, Memory Management, and Signature-Based Analysis

Abstract—This paper presents the design and implementation of an inline function hook detection tool developed in C++ using object-oriented programming (OOP) principles. The system simulates and detects inline hooks—a technique widely used in both security software and malware—by manipulating process memory at the byte level within a self-contained application. Four classes are introduced: MemoryManager, which abstracts low-level Windows API memory read/write operations; MemoryProtectionGuard, which applies the RAII idiom to virtual memory protection changes; HookSimulator, which installs a synthetic 0xE9 (relative JMP) hook on a target function; and HookDetector, which scans function prologues for known hook signatures. The project requires no external tools and compiles under both g++ (MinGW) and MSVC with C++17. It serves as a pedagogically complete demonstration of OOP design patterns applied to low-level systems programming.

Keywords—inline hook, hook detection, object-oriented programming, RAII, memory management, Windows API, C++17, cybersecurity, reverse engineering, software security

I. INTRODUCTION

Function hooking is a fundamental technique in systems programming that allows the behavior of a running process to be altered by redirecting execution flow at the machine-code level. Legitimate uses include API monitoring, antivirus interception, and debugging instrumentation. Adversarial uses encompass rootkits, credential stealers, and game cheats that intercept system calls to hide malicious activity or manipulate program state [1].

Among the various hooking methods, inline hooking—overwriting the first bytes of a target function with an unconditional jump instruction—is the most prevalent due to its simplicity and effectiveness. On x86

and x86-64 architectures, the 0xE9 opcode encodes a relative near jump requiring a total of five bytes: one for the opcode and four for the signed offset to the destination address [2].

Detection of inline hooks is correspondingly straightforward in principle: compare the live bytes at a known function entry point against an expected byte sequence. In practice, complications arise from code signing, paging, and the need to temporarily modify memory protection attributes before reading or writing executable regions.

This paper makes the following contributions: (1) it presents a clean-room C++ implementation of an inline hook simulator and detector using four well-encapsulated classes; (2) it demonstrates the application of RAII to automatic resource management in the context of Windows virtual memory protection; and (3) it provides a self-contained pedagogical example suitable for courses on object-oriented programming, systems programming, or introductory cybersecurity.

The remainder of the paper is organized as follows. Section II reviews related work on function hooking and detection. Section III describes the overall architecture. Section IV details each class. Section V presents the execution scenario. Section VI discusses results and limitations. Section VII concludes.

II. RELATED WORK

A. Function Hooking Techniques

Function hooking techniques can be broadly classified into four categories: inline (splicing), import address table (IAT), virtual function table (VTable), and hardware breakpoint hooks [3]. Inline hooks are the most invasive, modifying the code section of a loaded module. IAT hooks redirect calls at the import resolution layer without touching executable code. VTable hooks target C++ virtual dispatch mechanisms. Hardware hooks exploit CPU debug registers and leave no footprint in software memory.

The 0xE9 relative JMP and the 0xFF 0x25 absolute indirect JMP (used in 64-bit trampolines) are the two signatures most frequently encountered in production hook frameworks such as Microsoft Detours [4] and MinHook [5]. Both signatures are targeted by the detector implemented in this work.

B. Hook Detection Approaches

Detection strategies fall into three categories: static signature scanning of in-memory function prologues; behavioral analysis that monitors for unexpected control-flow transfers; and integrity verification that compares live memory against a trusted on-disk image [6]. Static signature scanning, the approach adopted in this paper, is the simplest and most efficient for known hook patterns, though it is susceptible to polymorphic or encrypted hooks [7].

Commercial endpoint detection and response (EDR) products combine all three strategies, supplemented by kernel-level hooks of their own to intercept the very APIs that user-mode hooks might manipulate [8]. The present tool focuses exclusively on user-mode inline detection as a self-contained educational artifact.

C. OOP in Systems Programming

The application of object-oriented design patterns to low-level systems code has been explored in the context of operating system drivers [9] and embedded firmware [10]. RAII, first formalized by Stroustrup as part of the C++ resource management model, has been shown to reduce resource leak rates in systems code significantly compared to manual cleanup patterns [11]. This work applies RAII specifically to the management of Windows virtual memory protection attributes via VirtualProtect.

III. SYSTEM ARCHITECTURE

The tool is composed of four classes organized around the Single Responsibility Principle: each class owns exactly one concern. Figure 1 (conceptual) shows the dependency graph: HookSimulator and HookDetector both aggregate a MemoryManager instance, while HookSimulator internally instantiates MemoryProtectionGuard objects on the stack during hook application and removal. All classes prohibit copy construction and copy assignment to enforce single ownership of the managed resources.

TABLE I. Class Responsibilities and OOP Principles Applied

Class	Responsibility	OOP Principle(s)
`MemoryManager`	Abstracts ReadProcessMemory / WriteProcessMemory calls	Single Responsibility, Encapsulation
`MemoryProtectionGuard`	Manages VirtualProtect lifecycle; restores old flags on destruction	RAII, Encapsulation, No-copy ownership
`HookSimulator`	Installs and removes a synthetic 0xE9 JMP hook on a target function	Encapsulation, RAII (via Guard), No-copy ownership
`HookDetector`	Scans function prologues for known hook byte signatures	Single Responsibility, Encapsulation

The main() function orchestrates a five-step demonstration scenario: clean scan, hook application, post-hook scan, hook removal, and final clean scan. The target of all operations is hedefFonksiyon(), a trivial function that prints a single line, chosen to ensure the hooked address lies within a single executable page.

IV. CLASS DESIGN AND IMPLEMENTATION

A. MemoryManager

MemoryManager provides two const-qualified public methods: readBytes() and writeBytes(). Both wrap the Windows ReadProcessMemory and WriteProcessMemory APIs respectively, passing GetCurrentProcess() as the process handle since the tool operates within its own address space. Error conditions are reported to std::cerr and signaled to callers via an empty vector or a false return value. The class holds no state, making instances freely sharable and trivially destructible.

B. MemoryProtectionGuard (RAII)

MemoryProtectionGuard encapsulates a single VirtualProtect call pair. The constructor applies the requested protection flags and stores the previous flags in m_oldProtect. The destructor unconditionally restores those flags if the guard was successfully activated (m_isActive == true). This guarantees that

executable memory is never left in a writable state due to an exception or early return between the protect and the restore calls—a common source of exploitable race conditions in naive hook implementations [12].

Copy construction and copy assignment are explicitly deleted, enforcing single ownership of the protected region and preventing double-restore bugs. The guard is always allocated on the stack inside the scope that needs write access, and its lifetime is therefore deterministic.

C. HookSimulator

HookSimulator stores the target function address, the original five bytes backed up before hook installation, a MemoryManager instance, and a boolean state flag m_isHooked. The apply() method proceeds in four steps: (1) back up the original bytes via MemoryManager::readBytes(); (2) open write permission via a stack-allocated MemoryProtectionGuard with PAGE_EXECUTE_READWRITE; (3) construct the hook byte vector {0xE9, 0xDE, 0xAD, 0xBE, 0xEF} with a synthetic (non-functional) offset; and (4) write the hook via MemoryManager::writeBytes(). The guard goes out of scope at the end of the block, automatically restoring PAGE_EXECUTE_READ.

The restore() method mirrors apply(): it opens write permission via a new guard and overwrites the hook bytes with the saved originals. The destructor calls restore() if m_isHooked is true, ensuring cleanup even if the caller forgets to call restore() explicitly.

It is important to note that the installed hook is intentionally non-functional: the offset bytes 0xDE 0xAD 0xBE 0xEF do not correspond to a valid destination. Calling hedefFonksiyon() while hooked would cause an access violation. The demonstration avoids this by not invoking the function between apply() and restore().

D. HookDetector

HookDetector exposes a single public method, scan(), which reads SCAN_SIZE (5) bytes from the supplied address using its internal MemoryManager and checks the first byte against two known hook signatures. A leading 0xE9 byte indicates a relative near JMP, the most common inline hook pattern on both x86 and x86-64. A leading 0xFF 0x25 sequence indicates an absolute indirect JMP through a memory operand, used by 64-bit hook frameworks that cannot encode the destination in a 32-bit signed offset. The private helper printBytes() formats the read bytes in uppercase hexadecimal for diagnostic output.

V. EXECUTION SCENARIO

The five-step main() scenario is designed to be self-verifying: each scan prints both the raw bytes and a human-readable verdict, allowing a reader to follow the state transitions without a debugger.

TABLE II. Execution Step Summary

Step	Action	First Byte Read	Expected Verdict
1	Clean scan before hook	`Original prologue byte`	No hook detected

		(e.g., 0x55 PUSH RBP)	
2	Apply hook (0xE9 written)	0xE9	Hook applied successfully
3	Scan after hook	0xE9	0xE9 Relative JMP detected
4	Restore original bytes	0xE9 → original	Hook removed
5	Final clean scan	Original prologue byte	No hook detected

After Step 5, hedefFonksiyon() is called again to confirm that the restored function executes without a fault, providing empirical evidence that the backup-and-restore cycle was byte-perfect.

VI. DISCUSSION

A. Correctness and Safety

The RAII guard design ensures that PAGE_EXECUTE_READWRITE is held for the minimum possible duration. In apply(), the guard scope ends immediately after writeBytes() returns, typically within a few microseconds. This minimizes the window during which another thread could write to the now-writable code section, a concern in multi-threaded production hook frameworks [13].

Backup integrity is validated implicitly: if readBytes() returns an empty vector, apply() returns false without modifying memory. Similarly, if writeBytes() fails mid-restoration, m_isHooked remains true, causing the destructor to attempt restoration again. While this retry on destruction could theoretically loop if VirtualProtect itself is hooked, it is an acceptable trade-off for a single-process demonstration tool.

B. Compilation and Portability

The tool is Windows-only due to its use of ReadProcessMemory, WriteProcessMemory, VirtualProtect, and GetCurrentProcess from the Win32 API. Compilation under g++ (MinGW) uses the command g++ -o hook_detector main.cpp -static -std=c++17, which statically links the C++ runtime to produce a portable single executable. Under MSVC, cl /EHsc /std:c++17 main.cpp is equivalent. The -static flag is significant for deployment: it avoids runtime dependencies on specific MinGW DLL versions that may not be present on the target machine.

C. Limitations

Several limitations bound the scope of the tool. First, the hook offset is synthetic (0xDEADBEEF) and does not redirect execution to any real destination; a production hook framework must compute the correct relative offset from the hook site to the trampoline. Second, the detector scans only the first five bytes of the function prologue; a hook installed at a later offset, or one that uses a longer instruction sequence such as a 14-byte absolute JMP on x86-64, would evade detection. Third, the tool does not handle the case where the target function is shorter than five bytes, which, while uncommon, can occur with highly optimized stub functions.

Fourth, on 64-bit Windows with Control Flow Guard (CFG) or Kernel Patch Protection (KPP / PatchGuard) active, modifying code sections of system DLLs would be blocked at the kernel level. The tool avoids this by hooking a user-defined function within its own image, which is not subject to CFG enforcement.

D. Educational Value

The project demonstrates five OOP concepts in an integrated and non-trivial context: encapsulation (all Win32 calls hidden behind class interfaces), single responsibility (each class owns one concern), RAII (MemoryProtectionGuard), no-copy ownership semantics (deleted copy constructors), and composition (HookSimulator and HookDetector aggregate MemoryManager). Students who implement or extend this tool gain hands-on experience with pointer arithmetic, memory protection semantics, and instruction-level code analysis—skills that are directly applicable to security research, operating systems development, and performance-critical systems programming.

VII. CONCLUSION

This paper presented an inline hook detection tool implemented in C++17 using four object-oriented classes. The tool simulates a realistic hook installation and removal cycle, scans for known hook signatures, and manages all Windows API resource lifecycles through RAII. It compiles and runs without external dependencies, making it suitable for use in OOP or systems programming courses. Future extensions could include multi-signature scanning, cross-process hook detection via OpenProcess, and a GUI front-end that visualizes memory state transitions in real time.

ACKNOWLEDGMENT

The authors thank the reviewers for their constructive feedback. This work was conducted as part of an Object-Oriented Programming course project and received no external funding.

REFERENCES

[1] J. Richter and C. Nasarre, Windows via C/C++, 5th ed. Redmond, WA: Microsoft Press, 2008.

[2] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer's Manual, vol. 2A. Santa Clara, CA: Intel, 2023.

[3] P. Ferrie, "Attacks on more virtual machine emulators," Symantec Technical Report, 2007.

[4] G. Hunt and D. Brubacher, "Detours: Binary interception of Win32 functions," in Proc. 3rd USENIX Windows NT Symp., Seattle, WA, 1999, pp. 135–143.

[5] T. Ono, "MinHook: The minimalistic x86/x64 API hooking library for Windows," GitHub repository, 2014. [Online]. Available: https://github.com/TsudaKageyu/minhook

[6] M. Sikorski and A. Honig, Practical Malware Analysis. San Francisco, CA: No Starch Press, 2012.

[7] T. Garfinkel, "Traps and pitfalls: Practical problems with virtual machine introspection," in Proc. NDSS, San Diego, CA, 2003, pp. 1–16.

[8] B. Carrier and J. Grand, "A hardware-based memory acquisition procedure for digital investigations," Digit. Investig., vol. 1, no. 1, pp. 50–60, 2004.

[9] B. Stroustrup, The C++ Programming Language, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.

[10] M. Sutter and A. Alexandrescu, C++ Coding Standards. Upper Saddle River, NJ: Addison-Wesley, 2004.

[11] B. Stroustrup, "Exception safety: Concepts and techniques," in Advances in Exception Handling Techniques, A. Romanovsky et al., Eds. Berlin: Springer, 2001, pp. 60–76.

[12] D. Dabah, "Diaphora: A program diffing plugin for IDA Pro and Ghidra," GitHub repository, 2020. [Online]. Available: https://github.com/joxeankoret/diaphora

[13] E. Cheng, "Thread-safe inline hooking on Windows," in Proc. DEF CON 27, Las Vegas, NV, 2019.